

RELATÓRIO 06 — PREPARAÇÃO DO PILOTO

M.I.N.D. — Muñdji Intelligent Neural Developer
Ferramenta de verificação e medição com modelos reais

Campo	Valor
Data	3 de Agosto de 2026
Commit	a143abf
Dimensão	5 ficheiros, 773 linhas
Suite	326 passam · 4 saltados (304 anteriores + 22 novos)
Natureza	Trabalho preparatório : o piloto ainda não correu, porque continua sem endpoint disponível

1. Porque é que isto existe

O MIND está inteiro e tem 326 testes a passar, mas **nunca correu com um LLM verdadeiro**. Essa continua a ser a maior lacuna do projecto, e não se fecha com mais código — fecha-se com uma instância a responder.

O que se podia fazer sem instância era tirar o atrito do caminho: garantir que, no dia em que houver endpoint, o piloto é **um comando** e não uma tarde de configuração. É isso que esta etapa entrega.

Concordaste com a ordem — piloto primeiro com a sandbox evolutiva desligada, para validar o ciclo base sem uma variável nova a confundir o diagnóstico, e só depois ligá-la. A ferramenta suporta as duas passagens sem alteração.

2. O que fiz

2.1 O caminho completo, do zero à medição

```
# 1. na instância alugada
./scripts/provisionar_vllm.sh Qwen/Qwen2.5-Coder-7B-Instruct
→ imprime o token gerado

# 2. na tua máquina – túnel
ssh -N -L 8000:127.0.0.1:8000 -p <porta> root@<host>

# 3. verificar antes de gastar GPU
python main.py verificar

# 4. correr e medir
python main.py piloto --max-tarefas 3
```

Componente	O que faz
<i>main.py verificar</i>	Testa a ligação a cada componente configurado sem correr ciclos . Reporta latência, o modelo que respondeu, e a razão exacta da falha. Sai com código 1 se algum falhar, para poder entrar num script.
<i>main.py piloto</i>	Verifica primeiro; se algo não responde, recusa arrancar. Depois corre as tarefas de referência medindo duração, iterações, taxa de aprovação, conformidade JSON e — com a sandbox ligada — quantos testes gerados são executáveis. Exporta CSV.
<i>config/piloto_tarefas.yaml</i>	Dez tarefas por dificuldade crescente, das triviais às de concorrência, com duas do domínio de pentest tooling. É o intervalo de 5-10 que o critério de selecção de modelo pede.

Componente	O que faz
<i>scripts/provisionar_vllm.sh</i>	Arranca um servidor vLLM na instância: gera token, suporta adaptadores LoRA, e escuta apenas em localhost.

3. Como fiz — quatro decisões

Decisão	Razão
Separei <i>verificar de piloto</i>	Um URL errado ou um token em falta descobre-se em segundos em vez de a meio de um ciclo com a GPU a contar ao minuto. O <i>piloto</i> chama a verificação primeiro e recusa arrancar contra um endpoint em baixo — o orçamento de GPU é real e não se queima a diagnosticar.
A conformidade JSON sai sem trabalho extra	Os desvios ao esquema já eram registados na SYNAPSE DB pelo <i>report_schema</i> , desde a entrega anterior. O piloto limita-se a contá-los. Não foi preciso instrumentação nova — a decisão de registar os desvios na altura certa é que tornou esta medição gratuita agora.
Uma tarefa que rebenta não interrompe as seguintes	Um piloto que pára na primeira falha não mede nada. Cada tarefa é isolada: a que falha fica registada com o erro, e passa-se à frente. É o oposto do que se quer num teste unitário, e o correcto aqui.
O servidor escuta só em localhost	A exposição faz-se por túnel SSH, não abrindo a porta. Um servidor de inferência aberto é uma GPU que estás a pagar para servir outra pessoa — e os teus prompts nas mãos dela. O token é exigido de qualquer forma, mas o túnel é a defesa que não depende de o token não vaziar.

4. Verificação

22 testes novos, e os dois casos que interessam foram também exercitados à mão contra a CLI real:

Cenário	Comportamento observado
Sem modelos configurados (estado actual)	Reporta 'sem modelo configurado' nos 8 componentes e sai com 0. Não tenta ligar a nada — confirmado por teste que falha se houver qualquer chamada HTTP.
Endpoint inexistente	Identifica o componente, o URL e a razão (<i>Connection refused</i>), e sai com 1.
Token errado (401)	Uma tentativa por componente, sem retry — um 401 não melhora com insistência.
Instância a reiniciar (erro transitório)	Retry com backoff, como em qualquer chamada a modelo.
Tarefa que rebenta a meio do piloto	Fica com status 'erro' e a razão; as seguintes correm na mesma.

5. O que não fiz — e porquê

Item	Razão
Não corri o piloto	Não há endpoint. É a razão de esta etapa ser preparatória e não conclusiva, e é por isso que não há um único número real neste relatório.
Não automatizei o túnel SSH	É configuração de infraestrutura, não de aplicação. Pôr o MIND a gerir túneis alargava-lhe o âmbito sem necessidade. O comando está documentado no cabeçalho do script.
Não integrei o piloto com o preenchimento automático do critério de selecção	O CSV tem os números, mas as fichas de <i>model_selection_criteria.md</i> pedem também juízo (licença, comunidade, adequação). Preenchê-las automaticamente daria a ilusão de um critério cumprido quando metade dele exige decisão humana.
Não escolhi modelos	O critério documentado exige pesquisa e teste-piloto antes de fixar qualquer <i>_MODEL</i> . Escolher agora seria saltar o processo que a própria especificação impõe.

6. O que vou fazer a seguir

Nada mudou no bloqueio, e continuo a precisar apenas de **três coisas: URL do servidor, token e id do modelo**. Com isso:

#	Passo	O que produz
1	verificar contra a instância	Confirma ligação, token e latência por componente. Minutos, não horas.
2	<i>piloto --max-tarefas 3</i> com a sandbox desligada	As primeiras medições verdadeiras do projecto: o ciclo chega ao fim? quanto demora? quantas iterações?
3	Piloto completo, 10 tarefas	Taxa de aprovação e conformidade JSON por modelo — o número que decide se um modelo serve
4	Ligar a sandbox evolutiva e repetir	Quantos dos testes gerados são executáveis. É a pergunta que a extensão levanta e que só um modelo real responde.
5	Aferir timeouts e preencher o critério de selecção	NEURON_TIMEOUT_SECONDS=60 e MUNDJI_SANDBOX_TIMEOUT=30 são palpites meus; passam a ser medidos

Nota sobre a entrega anterior

O *MIND_Prompt_Final_Sandbox.pdf* reenviado nesta sessão é byte a byte o mesmo que já tinha sido implementado (MD5 `facf5b2f...`), no commit `06a57a6`. Verifiquei antes de agir e não repeti trabalho. Se havia uma versão revista para enviar, o upload trouxe a antiga.

Relatório de uma etapa preparatória: descreve uma ferramenta testada contra respostas simuladas, que nunca foi exercitada contra um servidor de inferência real. Os números reais só existirão depois do passo 2 da secção 6.